به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین سوم

مدرس: دکتر محمد قدسی، مهدی صفرنژاد — انتشار: ۲۰ اسفند ۹۹

درخت

زمان تحویل این تمرین ۱۵ فروردین ۱۴۰۰ می‌باشد. سؤالات را با دقت بخوانید و روی همه آن‌ها وقت بگذارید. تمرین‌های تئوری تحویل گرفته نمی‌شوند اما از آن‌ها سؤالات کوییز مشخص می‌شود. بنابراین روی سؤالات به خوبی فکر کنید و در کلاس‌های حل تمرین مربوطه شرکت کنید.

سؤال ۱. در این مسئله می‌خواهیم شیوه‌ای در مورد پیاده‌سازی پیمایش میان‌ترتیب درخت دودویی جست‌وجو [1] را بررسی و تحلیل کنیم.

یکی از روش‌های پیاده‌سازی میان‌ترتیب، استفاده از دو تابع به نامی های $tree_min$ و $inorder_successor$ است. تابع اول یک ریشه ورودی گرفته و عنصر کمینه زیردرخت نظیر آن (چپ‌ترین گره) را بر می‌گرداند. تابع دوم یک گره را ورودی گرفته و گره بعدی در پیمایش میان‌ترتیب را پیدا می‌کند. پیاده‌سازی آن به شکل زیر است:

---

procedure $inorder_successor(x)$

  if $x.right \neq$ null **then return** $tree_min(x.right)$

  $y = x.parent$

  while $y \neq$ null **and** $x == y.right$ **do**

    $x = y$

    $y = y.p$

  return y

توجه الگوریتم بالا معادل الگوریتم صفحه 83 اسلایدهاست.

آ. فرض کنید T یک درخت دودویی جست‌وجو با عناصر متمایز باشد. نشان دهید که اگر x زیردرخت راست نداشته باشد و آخرین عنصر در پیمایش میان‌ترتیب هم نباشد، آن‌گاه عنصر بعدی در پیمایش میان‌ترتیب بعد از x، پایین‌ترین جدی از x خواهد بود که فرزند سمت چپ آن هم جد x باشد. به بیان دیگر، این سوال قصد اثبات درستی تابع $inorder_successor$ را دارد.

ب. یک راه برای چاپ پیمایش میان‌ترتیب درخت دودویی جست‌وجو، این است که ابتدا یک بار تابع $tree_min$ را روی ریشه کل درخت صدا کرده و پس از آن، $n-1$ بار تابع $inorder_successor$ را روی خروجی مرحله قبل صدا کنیم. این روش به وضوح درست است. مرتبه زمانی اجرای این الگوریتم را بدست آورید.

---

[1] inorder

پاسخ: فرض کنید راسی که مسئله خواسته است را با y نمایش دهیم. ابتدا نشان می‌دهیم که y باید جد x باشد. اگر این طور نباشد، فرض کنید z اولین جد مشترک x و y باشد. با توجه به این که x فرزند سمت راست هم ندارد طبق خاصیت درخت دودویی جست‌وجو باید داشته باشیم $x < z < y$. در این صورت y نمی‌تواند عنصر بلافاصله بعد از x در پیمایش میان‌ترتیب باشد. در نتیجه فرض خلف باطل است و y باید جد x باشد.

حال توجه کنید که $y.left$ باید جد x باشد، زیرا اگر این طور نباشد، $y.right$ جد x خواهد بود. اگر این طور باشد یعنی $x > y$ که باز هم نتیجه می‌دهد که y نمی‌تواند عضو بعدی در پیمایش میان‌ترتیب باشد. در نهایت نشان می‌دهیم که y باید پایین‌ترین جد x باشد که فرزند چپ آن هم جد x است. فرض کنید y پایین ترین جد با این خاصیت نباشد. در آن صورت z با این شرایط وجود دارد. این بدین معنی خواهد بود که z خودش در زیردرخت سمت چپ y است که یعنی $z < y$ که این هم با نحوه پیماش میان ترتیب و این که y عضو بعدی در این پیمایش بوده است، تناقض خواهد داشت.

برای تحلیل زمانی الگوریتم، نشان‌می‌دهیم که این الگوریتم از $\Theta(n)$ است. برای این کار نشان می‌دهیم که الگوریتم پیمایش میان‌ترتیب به شکل گفته شده، هر یال را دو بار پیمایش می‌کند و از آن جایی که تعداد یال ها یکی کمتر از راس‌هاست، الگوریتم در کل از $\Theta(n)$ خواهد بود.

برای اثبات این موضوع یک راس را به نام x در درخت دودویی جست‌وجو در نظر بگیرید. حال یال بین x و $x.parent$ را در نظر بگیرید. این یالیک بار در هنگامی که $inorder_successor$ روی $x.parent$ فراخوانی می‌شود پیموده شده و بار دیگر، زمانی که این تابع روی بزرگترین عنصر موجود در زیردرخت با ریشه x صدا زده می‌شود، برای یافتن جد آن پیموده خواهد شد. از آن جایی که همه یال‌ها با این استدلال تنها دو بار پیموده می‌شوند و یک هزینه اولیه هم برای $tree_min$ وجود دارد که تاثیری در مرتبه زمانی نهایی الگوریتم نمی‌گذارد. در نتیجه زمان اجرا از $O(n)$ خواهد بود و همچنین از آن‌جایی که اندازه خروجی n است، از $\Omega(n)$ هم هست. در نتیجه این الگوریتم از $\Theta(n)$ است.

سؤال ۲. پیاده‌سازی درخت دودویی جست‌وجو با عناصر تکراری چالش‌هایی از نظر زمانی ایجاد می‌کند. در این مسئله می‌خواهیم به بررسی این موضوع بپردازیم. فرض کنید عمل درج ساده ما مشابه تابع NR_BST_INSERT در صفحه ۸۵ اسلایدهاست.

آ. در صورتی که n بار پشت سرهم بخواهیم یک داده یکسان را در درختی که در ابتدا خالی است درج کنیم، مرتبه زمانی این کار به چه صورت خواهد بود؟

در ادامه مسئله قصد داریم با چک کردن شرط یکسان بودن کلیدها پیش از خطوط 6 و 14 در الگوریتم صفحه ۸۵، به شکل خاصی حالت برابری کلیدها را پیاده سازی کنیم. برای هر یک از حالت‌هایی که در زیر پیشنهاد می‌شود، پیچیدگی زمانی الگوریتم را برای درج n عنصر یکسان تحلیل کنید.

ب. یک مقدار flag صفر و یکی در هر عنصر نگه داریم.آن گاه بسته به این که مقدار این flag برابر True یا False باشد، در خط ۷ الگوریتم $p \leftarrow left[p]$ یا $p \leftarrow right[p]$ را اجرا کنیم. مقدار این flag در هر بار گذر از یک راس در هنگام درج مقداری برابر با آن راس در درخت، از True به False یا از False به True تغییر می‌کند.

پ. لیستی از گره‌هایی که مقادیر برابری با گره x دارند را در آن نگه داریم و هر بار هنگام درج مقداری برابر با آن گره، آن مقدار را به آن لیست اضافه کنیم.

ت. برای این قسمت مشابه بخش ب عمل می‌کنیم با این تفاوت که دیگر flag ای در نظر نمی‌گیریم و به طور کاملا تصادفی (با احتمال برابر) به سمت چپ یا راست می‌رویم. برای این قسمت هم بدترین حالت را مشخص کنید و هر به صورت شهودی حالت میانگین را تحلیل کنید.

پاسخ:

برای قسمت اول، از آن جایی که شرط خط 14 همواره false می‌شود، هر عنصر به راست‌ترین سمت درخت اضافه می‌شود. در نتیجه زمان اجرا به صورت $\sum_{i=1}^{n} i \in \Theta(n^2)$ خواهد بود.

برای قسمت دوم سوال، این روش منجر می‌شود که هر کدام از زیردرختان تنها به اندازه 1 واحد در ارتفاع با هم فاصله داشته باشند. در نتیجه ارتفاع از $\Theta(\log(n))$ خواهد بود. در نتیجه کل زمان اجرا به صورت $\sum_{i=1}^{n} \log(n) \in \Theta(n \log(n))$ خواهد بود.

در قسمت سوم، با فرض این که صرفا n درج برابر پشت سر هم داریم، ارتفاع درخت همواره 0 باقی می‌ماند و از آن جایی که درج ساده در لیست هم در زمان ثابت $\Theta(1)$ امکان پذیر است، کل زمان اجرا به صورت خط و از $\Theta(n)$ خواهد بود.

برای قسمت چهارم، بدترین حالت این است که هر دفعه سمت راست به صورت تصادفی انتخاب شود و در این حالت مشابه بخش اول و $\Theta(n^2)$ می‌شود. در حالت میانگین به شکل شهودی، نصف اوقات سمت راست و نصف اوقات سمت چپ را انتخاب می‌کند و در نتیجه درخت تقریبا متعادل از دو سمت گسترش پیدا می‌کند. در نتیجه مشابه بخش دوم می‌شود و در حالت میانگین به صورت $\Theta(n \log(n))$ خواهد بود.

سؤال ۳. در این سوال می‌خواهیم داده‌ساختاری شبیه هرم [۲] را پیاده‌سازی و تحلیل کنیم. با این تفاوت که برخلاف Heap که هر راس حداکثر دو فرزند داشت، در این جا هر راس می‌توانند حداکثر d فرزند داشته باشد. به این داده ساختار هرم d ـ تایی می‌گوییم.

آ. چنین هرمی که هر راس‌آن حداکثر d فرزند دارد را چه طور در آرایه ذخیره می‌کنید؟ برای حل این سوال کافیست توابعی که اندیس فرزند k ام عنصر i ام و والد عنصر i ام را برمی‌گردانند را بنویسید.

ب. ارتفاع یک هرم d ـ تایی با n عضو چند است؟

پ. یک پیاده‌سازی بهینه برای تابع $Extract_Max$ در هرم بیشینه d ـ تایی ارائه کنید و آن را برحسب n و d از لحاظ زمانی تحلیل نمایید.

ت. یک پیاده‌سازی بهینه برای تابع $Insert$ در هرم بیشینه d ـ تایی ارائه کنید و آن را برحسب n و d از لحاظ زمانی تحلیل نمایید.

ث. یک پیاده‌سازی بهینه برای تابع $Increase_Key(A, i, k)$ در هرم بیشینه d ـ تایی ارائه کنید. این تابع در صورتی که عنصر i ام بزرگ‌تر از k باشد خطا بر می‌گرداند و در غیر این صورت مقدار $A[i]$ را برابر k قرار داده و سپس هرم را آپدیت می‌کند تا خاصیت هرم بیشینه بودن آن حفظ شود. الگوریتم را برحسب n و d از لحاظ زمانی تحلیل نمایید.

پاسخ:

برای قسمت اول توابع را به این شکل پیاده سازی می‌کنیم.

---

procedure $Parent(i)$ **return** $\lfloor \frac{i}{d} \rfloor$

procedure $Child(i, k)$ **return** $d(i-1)+1+k$

برای قسمت دوم جواب $\Theta(\log_d(n))$ است.

[۲] heap

برای قسمت سوم دو تابع $HeapExtractMin$ و $DMaxHeapify$ را پیاده‌سازی می‌کنیم.

```
procedure HeapExtractMin(A)
    max = A[1]
    A[1] = A[A.HeapSize]
    A.HeapSize = A.HeapSize − 1
    DMaxHeapify(A, 1)
procedure DMaxHeapify(A, i)
    largest = i
    for k = 1 to d do
        if Child(i, k) ≤ A.HeapSize and A[Child(i, k)] > A[i] then
            if A[Child(i, k)] > largest then
                largest = A[Child(i, k)]
    if largest ≠ i then
        swap A[i] with A[largest]
        DMaxHeapify(A, largest)
```

برای تحلیل زمانی، تابع $HeapExtractMin$ تعدادی عملیات با مرتبه زمانی ثابت و یک بار فراخوانی $DMaxHeapify$ است. برای این تابع، تعداد دفعات فراخوانی به عمق هرم بستگی دارد و در هر بار اجرا، یک حلقه d تایی داریم پس پیچیدگی زمانی آن از $O(d\log_d(n))$ است.

برای قسمت بعدی تابع $Insert$ را پیاده‌سازی می‌کنیم.

```
procedure Insert(A, key)
    A.HeapSize = A.HeapSize + 1
    A[A.HeapSize] = key
    i = A.HeapSize
    while i > 1 and A[Parent(i)] < A[i] do
        swap A[i] with A[Parent(i)]
        i = Parent(i)
```

مرتبه زمانی اجرای این الگوریتم وابسته به عمق هرم است یعنی این الگوریتم از $O(\log_d(n))$ است.

برای قسمت آخر تابع $IncreaseKey$ را به صورت زیر پیاده سازی می‌کنیم:

```
procedure IncraseKey(A, i, k)
    if k < A[i] then
        Throw Error
    A[i] = k
    while i > 1 and A[Parent(i)] < A[i] do
        swap A[i] with A[Parent(i)]
        i = Parent(i)
```

مشابه بخش قبل مرتبه زمانی اجرای این الگوریتم وابسته به عمق هرم است یعنی این الگوریتم از $O(\log_d(n))$ است.

سؤال ۴. رشته‌ای به طول n داده شده، الگوریتمی ارائه کنید که تعداد زیررشته‌های متمایز این رشته را در زمان $O(n^2)$ بدست آورد.

برای مثال رشتهٔ $ababa$ شامل ۱۰ زیررشته متمایز است.

پاسخ: برای انجام این کار تمامی پسوند [3] های رشته را در نظر می‌گیریم و با آن‌ها یک درخت ترای می‌سازیم و حالا تعداد گره‌های درخت حاصل، جواب مسئله خواهد بود.

برای استدلال درستی راه حل به دو نکته توجه داشته باشید.

۱. هر مسیر از ریشه درخت ترای به هر گره از آن نشان‌دهنده یک پیشوند [4] از رشته‌های وارد شده به درخت است. در اینجا این رشته‌ها پسوندهای رشته ورودی هستند پس می‌توان گفت هر گره از درخت نمایان‌گر یک پیشوند از پسوندهای رشته ورودی است.

۲. هر زیررشته از رشته ورودی، یک پیشوند از یکی از پسوندهای رشته ورودی است.

با این دو نکته درستی راه حل ما نتیجه می‌شود. پیچیدگی زمانی راه حل با توجه به استفاده از درخت ترای از $O(n^2)$ خواهد بود اگر n اندازه رشته ورودی باشد.

سؤال ۵. دنباله‌ای تهی از اعداد داریم. هر بار یکی از دو عملیات زیر از ما خواسته می‌شود و باید آن‌ها را در $O(\log_2(n))$ انجام دهیم که n تعداد عناصر موجود در دنباله در زمان اجرای آن عمل است. روشی برای انجام این کار ارائه کنید. عملیات:

۱. عنصر جدیدی به دنباله اضافه کنیم.

۲. میانهٔ عناصر موجود در دنباله را اعلام کنیم.

منظور از میانه در این سوال عنصر با اندیس $\lceil \frac{n}{2} \rceil$ در دنباله مرتب صعودی است، با فرض آنکه عناصر با شروع از ۱ شماره‌گذاری شوند. پس یعنی میانهٔ دنباله ۳، ۲، ۱ عدد ۲ است و میانهٔ دنباله ۱۶، ۸، ۴، ۲ عدد ۴ است. (دقت کنید که مجاز نیستیم ابتدا تمام درخواست‌ها را دریافت کنیم و سپس همه رو پاسخ دهیم بلکه باید بعد از هر پرسش، یعنی عملیات نوع دوم، میانه را اعلام کنیم)

[3] suffix
[4] prefix

پاسخ: در ابتدا یک max_heap و یک min_heap در نظر می‌گیریم. اگر درخواست نوع یک دریافت کردیم، اگر اندازه max_heap با min_heap برابر بود عدد را در max_heap وارد می‌کنیم و در غیر این صورت عدد را در min_heap وارد می‌کنیم. و اگر درخواست نوع دو دریافت کردیم، عنصر ریشه‌ی max_heap را اعلام می‌کنیم.

واضح است که تمامی عملیات‌ها از $O(log_2(n))$ پشتیابنی می‌شوند.

سؤال ۶. درختی دودویی را در نظر بگیرید که در آن هر گره یا دقیقا دو فرزند دارد یا فرزندی ندارد و به گره‌های آن اعداد ۱ تا n بدون ترتیب خاصی نسبت داده شده است. پیش‌ترتیب و میان‌ترتیب این درخت به ما داده شده‌است. روشی برای بدست آوردن خود درخت ارائه کنید و ثابت کنید با داشتن پیش‌ترتیب و میان‌ترتیب، درخت به صورت یکتا مشخص می‌شود.

پاسخ: اگر بتوانیم ریشه درخت و فرزندانش را به طور یکتا مشخص کنیم و پیش‌ترتیب و میان‌ترتیب زیردرخت‌های فرزندانش را هم مشخص کنیم، مسئله به کمک استقرای قوی روی تعداد رئوس درخت حل می‌شود. با توجه به تعریف پیش‌ترتیب، عنصر اول پیش‌ترتیب، همان ریشه درخت است. ریشه را در میان‌ترتیب در نظر بگیرید. با توجه به اینکه می‌دانیم هر گره یا دو فرزند دارد یا فرزندی ندارد، دنباله‌ی عناصر قبل از ریشه در صورت وجود، الزاما میان‌ترتیب فرزند چپ ریشه است و همچنین عناصر بعد از ریشه در صورت وجود، الزاما میان‌ترتیب فرزند راست ریشه است. پس حالا ما تمامی گره‌هایی که باید در زیردرخت سمت راست (یا چپ) ریشه باشند را داریم. با توجه به اینکه عناصر هر زیردرخت را مشخص کرده‌ایم میان‌ترتیب و پیش‌ترتیب زیردرخت راست و چپ هم به سادگی قابل استخراج هستند و حال که این موارد را داریم می‌توانیم به صورت بازگشتی عمل کنیم و زیردرخت‌های راست و چپ را بسازیم و بدین ترتیب مسئله حل می‌شود. برای اثبات یکتا بودن جواب هم دقیقا به همین ترتیب عمل می‌کنیم و از استقرای قوی استفاده می‌کنیم.

سؤال ۷. اعداد روی یک درخت جستجوی دودویی را با یک روش بهینه طوری روی درخت قرار دهید که درخت تبدیل به یک هرم بیشینه شود.

پاسخ: ابتدا روی درخت جست و جوی دوودیی $inorder$ پیمایش می کنیم و اعضای خوانده شده را در یک آرایه می‌ریزیم به این ترتیب مقادیر مرتب شده موجود در درخت را خواهیم داشت. سپس روی درخت این بار پیمایش $postorder$ می‌کنیم و در رسیدن به هر گره عنصری را به ترتیب از ابتدای آرایه خوانده و در درخت می‌ریزیم و به این ترتیب زمانی که به سر هرم برسیم قطعا بزرگ‌ترین عدد را در آرایه داریم که در سر هرم قرار می‌دهیم. همچنین همه اعضا موجود در هرم به وجود آمده بزرگ‌تر از نواده های خود و کوچک‌تر از اجداد خود خواهند بود.

سؤال ۸. یک آرایه از اعداد و عدد a داده شده است. می‌دانیم هر عدد تا جایگاه اصلی خود در آرایه حداکثر a واحد فاصله دارد. الگوریتمی از $\Omega(nloga)$ با $\Omega(a)$ حافظه اضافی ارائه دهید که آرایه را سورت کند.

پاسخ: برای این کار از یک هرم کمینه استفاده می‌کنیم. در این هرم $a+1$ عنصر اول را قرار می‌دهیم. سپس عنصر سر هرم را خارج کرده و در آرایه نهایی قرار می‌دهیم و عنصر بعدی را در هرم قرار می‌دهیم. قرار دادن عنصر جدید در مکان درستش در هرم $\Omega(loga)$ زمان می برد. همین کار را برای همه اعضای باقی مانده تکرار می کنیم. به این ترتیب زمان اجرای کل الگوریتم $\Omega(a)$ برای ساخت اولیه هرم و $(n-a)\Omega(n)$ می‌شود که به طور میانگین برابر $\Omega(nloga)$ است.

موفق باشید